



Bhartiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Department of Computer Engineering

Earthquake Risk Analyst

By

Anish Amit Joshi(2023300096)
Shivam Santosh Kadam (2023300099)
Ruchir Rajesh Kadam (2023300098)

Guided by
Abhijeet Salunke SIR

Course Project
Python for DATA SCIENCE (S.Y.)

Abstract

This project aimed to develop a predictive model for earthquake magnitudes utilizing historical seismic data, specifically focusing on the relationships between geographical and geological features. The dataset comprised 103,973 rows and 23 columns, including essential features such as latitude, longitude, depth, and magnitude.

Various machine learning algorithms were employed, including Linear Regression, Polynomial Regression, Random Forest, Gradient Boosting, and XGBoost, with hyperparameter tuning conducted via RandomizedSearchCV and BayesSearchCV to enhance model performance. The results demonstrated that both Gradient Boosting and XGBoost achieved high accuracy within a 0.5 magnitude threshold, with Gradient Boosting emerging as the optimal model due to its superior test accuracy.

The analysis highlighted the importance of relevant feature selection and the balance between bias and variance in model performance. However, the project faced limitations, including a dataset predominantly clustered around the 5 MW range, which may restrict the model's generalizability.

Future work suggests incorporating additional geological and environmental features, expanding the dataset to include a wider range of magnitudes, and exploring spatial models to capture regional variations in seismic activity. Overall, this project contributes valuable insights into earthquake magnitude prediction, emphasizing the need for continuous improvement in model robustness and accuracy.

1 Introduction

Problem Statement

Earthquakes pose significant risks to human life, infrastructure, and the environment, making it imperative to develop effective prediction models. Despite advancements in seismology, accurately predicting the timing and magnitude of earthquakes remains a formidable challenge. This project addresses the task of predicting earthquake magnitudes based on historical seismic data, focusing on global earthquake occurrences from 1900 to 2024. The scope of the project encompasses data collection, preprocessing, model development, and evaluation, with the aim of forecasting potential earthquake magnitudes for the years 2025 to 2030.

Objective

The primary goal of this project is to predict the magnitude of future earthquakes using machine learning techniques. Specifically, we aim to employ a Gradient Boosting algorithm to analyze historical earthquake data, allowing us to generate reliable predictions based on geographic coordinates (latitude and longitude), depth, and other relevant features. By doing so, we aspire to contribute valuable insights that can be used in disaster preparedness and response planning.

Motivation

The motivation behind this project stems from the urgent need for improved earthquake prediction methods. Accurate predictions can significantly enhance disaster preparedness, enabling communities to take preventive measures that mitigate damage and save lives. With millions of people residing in earthquake-prone areas, understanding potential seismic activity is crucial for effective risk management and response strategies. By harnessing the power of machine learning, this project seeks to advance the field of seismology and enhance public safety.

Outline

This report is structured as follows:

- Section 2 provides a detailed review of related work and existing methodologies in earthquake prediction.
- Section 3 outlines the data collection and preprocessing steps undertaken to prepare the dataset for analysis.
- Section 4 describes the machine learning algorithms utilized, including the rationale for selecting the Gradient Boosting algorithm.

- Section 5 presents the results of the model evaluation, including performance metrics and validation techniques.
- Section 6 discusses the implications of the findings, potential limitations, and avenues for future research.
- Finally, Section 7 tells references used

2 Dataset Description

2.1 Data Source

Source: Kaggle Dataset - Recent Earthquakes

URL: <https://www.kaggle.com/datasets/usamabuttar/significant-earthquakes/codeKaggle>
Earthquake Dataset

Size: The dataset contains **103,973 rows** and **23 columns**, indicating a substantial collection of data.

Type of Data: The dataset includes earthquake-related measurements and statistics.

Features:

- **Time:** The timestamp of the earthquake event.
- **Latitude:** The latitude coordinate of the earthquake location.
- **Longitude:** The longitude coordinate of the earthquake location.
- **Depth:** The depth of the earthquake in kilometers.
- **Magnitude (Mag):** The magnitude of the earthquake.
- **Magnitude Type (MagType):** The type of magnitude measurement (e.g., mw, ms, fa).
- **Number of Stations (Nst):** The number of seismic stations used to locate the earthquake.
- **Azimuthal Gap (Gap):** The azimuthal gap, which is the largest angle between stations used to locate the earthquake.
- **Distance to Nearest Station (Dmin):** The distance to the nearest station from the earthquake epicenter.
- **Updated:** The date the earthquake data was last updated.
- **Place:** The location or place name of the earthquake.
- **Type:** The type of earthquake (e.g., earthquake).
- **Horizontal Error:** The horizontal error in the earthquake location.
- **Depth Error:** The depth error in the earthquake location.
- **Magnitude Error (MagError):** The error in the earthquake magnitude.
- **Magnitude Number of Stations (MagNst):** The number of stations used to determine the earthquake magnitude.
- **Status:** The status of the earthquake data (e.g., reviewed).
- **Location Source:** The source of the earthquake location data.
- **Magnitude Source:** The source of the earthquake magnitude data.

2.2 Preprocessing

2.2.1 Feature Selection

Data cleaning techniques were employed to enhance the dataset's quality. Irrelevant columns, such as 'Unnamed: 0', 'status', and other non-essential features, were removed.

The 'place' variable had 437 missing values; if not needed, this column could be deleted, or missing entries could be filled based on geographical coordinates or other parameters using machine learning algorithms.

	time	latitude	longitude	depth	mag	magType	type	horizontalError	depthError	magError	magNst
0	1900-10-09 12:25:00+00:00	57.0900	-153.4800	NaN	7.86	mw	earthquake	NaN	NaN	NaN	NaN
1	1901-03-03 07:45:00+00:00	36.0000	-120.5000	NaN	6.40	ms	earthquake	NaN	NaN	NaN	NaN
2	1901-07-26 22:20:00+00:00	40.8000	-115.7000	NaN	5.00	fa	earthquake	NaN	NaN	NaN	NaN
3	1901-12-30 22:34:00+00:00	52.0000	-160.0000	NaN	7.00	ms	earthquake	NaN	NaN	NaN	NaN
4	1902-01-01 05:20:30+00:00	52.3800	-167.4500	NaN	7.00	ms	earthquake	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...	...
103968	2024-09-28 12:32:38.564000+00:00	25.8513	141.7033	86.547	5.20	mb	earthquake	10.76	7.489	0.053	119.0
103969	2024-09-28 03:04:15.083000+00:00	41.8708	-126.8324	10.000	5.10	mww	earthquake	9.01	2.014	0.044	49.0
103970	2024-09-27 21:56:48.177000+00:00	-22.9451	-177.1964	165.627	5.20	mww	earthquake	11.02	5.358	0.073	18.0
103971	2024-09-27 18:35:22.035000+00:00	-41.2103	-90.1998	10.000	5.00	mww	earthquake	12.18	1.856	0.080	15.0
103972	2024-09-27 14:58:07.869000+00:00	-19.9659	169.2946	99.540	5.10	mww	earthquake	8.32	6.442	0.071	19.0

Figure 1: Data Cleaning Techniques

	0
time	0
latitude	0
longitude	0
depth	285
mag	0
magType	0
type	0
horizontalError	81627
depthError	49713
magError	67004
magNst	59981

Figure 2: Null values

2.2.2 Handling Null Values

In the preprocessing phase, we addressed missing data identified in various columns. Columns with excessive null values, such as 'horizontalError', 'depthError', 'magError', and 'magNst', were dropped.

For the 'depth' column, we utilized a custom function for imputation, filling in missing entries with the mean or median of existing values, preserving the overall data distribution.

```
1 def filterForCol(dataset, col):
2     data = dataset
3     data_mean_imp = data
4     data_mean_imp.loc[data_mean_imp[col].isnull(), col] =
5         data_mean_imp[col].mean()
6     data_median_imp = data
7     data_median_imp.loc[data_mean_imp[col].isnull(), col] =
8         data_mean_imp[col].median()
9     data_dropped_rows = data
10    data_dropped_rows = data_dropped_rows.loc[data_mean_imp[col].
11        notnull()]
12
13    t0 = trifacta(data[col])
14    t1 = trifacta(data_mean_imp[col])
15    t2 = trifacta(data_median_imp[col])
16    t3 = trifacta(data_dropped_rows[col])
17
18    T1 = t1 - t0
19    T2 = t2 - t0
20    T3 = t3 - t0
21
22    if T1 <= T2 and T1 <= T3:
23        return data_mean_imp
24    elif T2 <= T3:
25        return data_median_imp
26    else:
27        return data_dropped_rows
28
29 df = filterForCol(df, 'depth')
```

2.2.3 Standardizing the Dataset

We found that the 'magtype' variable contained different units (e.g., mw, ms, fa), so we standardized these to a single unit.

Additionally, we converted time into a standard datetime format for consistency.

```
array(['mw', 'ms', 'fa', 'ml', 'mint', 'mb', 'lg', 'uk', 'mh', 'mwc',
      'md', 'mb_lg', 'mc', 'ma', 'mwb', 'mww', 'mwr', 'mlg', 'm', 'Md',
      'ML', 'ms_20', 'mwp', 'Mi', 'Mb', 'ml(texnet)'], dtype=object)
```

Figure 3: Standardization Process

Overview of Scaling Methods and Their Impact on Overfitting

In this analysis, we have employed various scaling techniques from the `sklearn.preprocessing` module, including `StandardScaler`, `MinMaxScaler`, `RobustScaler`, `MaxAbsScaler`, and `QuantileTransformer`. Each scaler has a distinct impact on model performance and overfitting:

- **StandardScaler:** This scaler standardizes features by removing the mean and scaling to unit variance. It works well if the data is normally distributed and is often a default choice; however, it may not significantly mitigate overfitting on its own.
- **MinMaxScaler:** This method scales the data to a fixed range, typically $[0, 1]$. While it is effective in many scenarios, it may not reduce overfitting if the dataset contains outliers, as these can distort the scaling range.
- **RobustScaler:** This scaler is particularly effective in reducing overfitting caused by outliers. It utilizes the median and the interquartile range (IQR), making it robust to extreme values and thus a good option when dealing with datasets that have outliers.
- **MaxAbsScaler:** This technique scales each feature by its maximum absolute value, making it suitable for sparse data. However, it has minimal impact on overfitting, as it does not address the distribution of the data.

- **QuantileTransformer (Normal)**: This method transforms features to follow a normal distribution, which can help reduce overfitting, especially when the data is skewed or non-Gaussian.

After evaluating the impact of each scaling method on overfitting, we conclude that the **RobustScaler** and **QuantileTransformer** are the best choices for addressing excessive overfitting. However, we ultimately selected the **QuantileTransformer** due to its effectiveness in transforming features to a normal distribution, thereby enhancing model performance.

2.3 Data Exploration

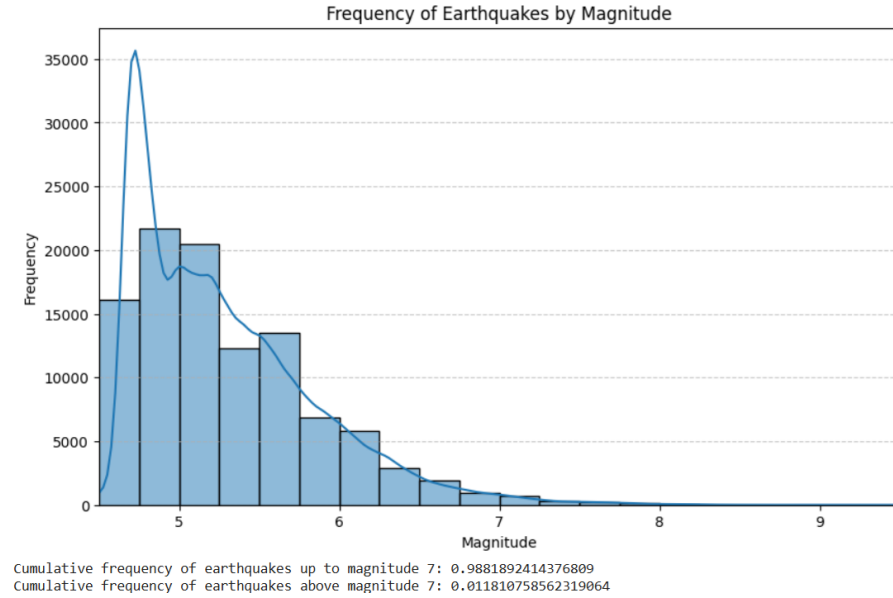


Figure 4: Earthquake Occurrences by Magnitude

The graph illustrates a significant trend in earthquake occurrences based on magnitude. The graph is highly skewed to the right. It is evident that earthquakes with magnitudes up to 7 have a high frequency of occurrence, indicating that smaller magnitude earthquakes are more likely to happen. In contrast, as we move beyond a magnitude of 7, the probability of an earthquake occurring diminishes significantly, suggesting that such high-magnitude events are relatively rare. This pattern highlights the importance of preparedness for lower-magnitude earthquakes while also recognizing the rarity of more severe seismic events.

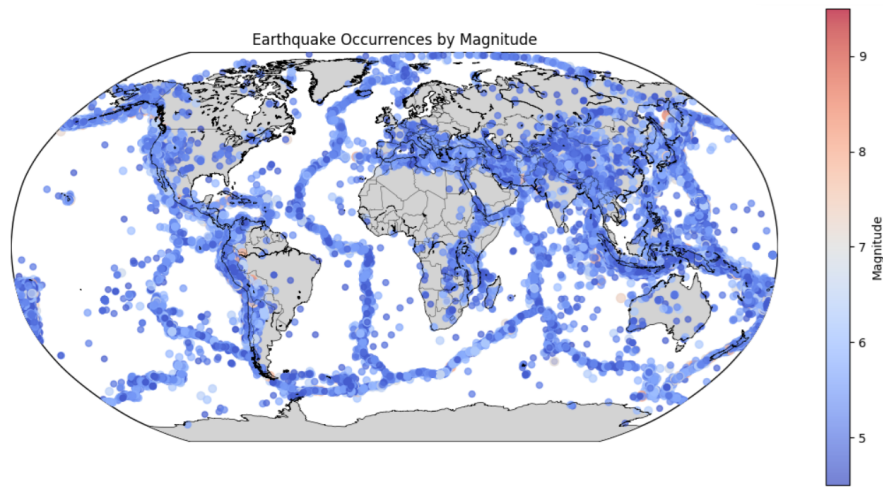


Figure 5: Key Observations of Earthquake Patterns

Key Observations:

1. Most earthquakes occur along tectonic plate boundaries, especially the Pacific "Ring of Fire."
2. Subduction zones show high earthquake frequency, such as along the American west coast and Indonesia.
3. Stronger earthquakes often occur where multiple plates collide, like the Himalayas and Japan Trench.
4. Some earthquakes appear within continents due to intraplate stresses or internal fault lines.
5. Oceanic regions, especially around the Pacific, have more seismic activity than continental interiors.
6. Regions like central Africa and Australia show few earthquakes, indicating tectonic stability.

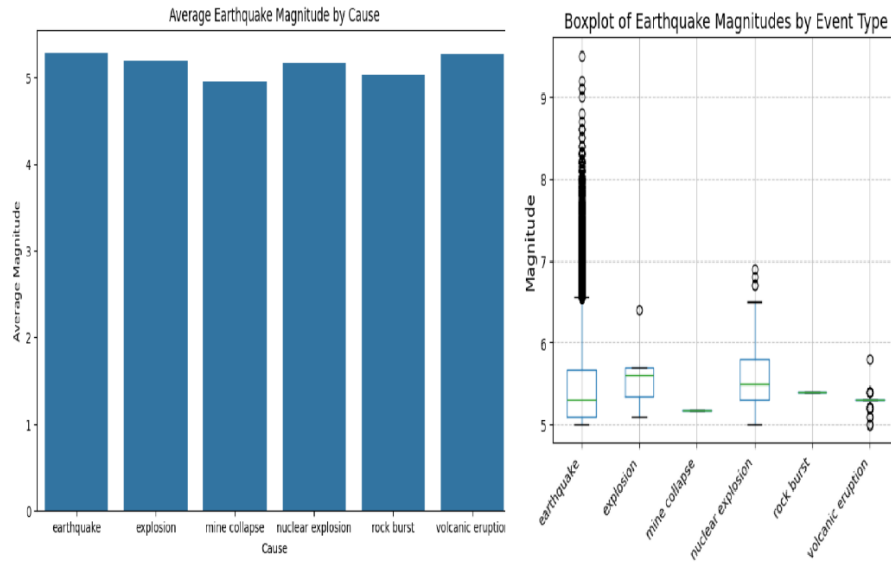


Figure 6: Comparison of Natural and Human-Caused Seismic Events

Looking at both graphs together reveals that while natural earthquakes demonstrate the highest variability in magnitude (ranging from 5 to 9 on the Richter scale) and contain the most extreme outliers, their average magnitude (around 5.3) is surprisingly similar to other seismic events like explosions and nuclear tests. Human-caused events such as explosions and nuclear tests show more consistent and controlled magnitude ranges, while natural phenomena like mine collapses, rock bursts, and volcanic eruptions generally produce lower and more predictable magnitudes. This pattern suggests that although natural earthquakes have the potential for more catastrophic events (as shown by their high-magnitude outliers), the typical seismic impact across different types of events remains relatively comparable, with human-induced seismic activities being more predictable in their magnitude ranges.

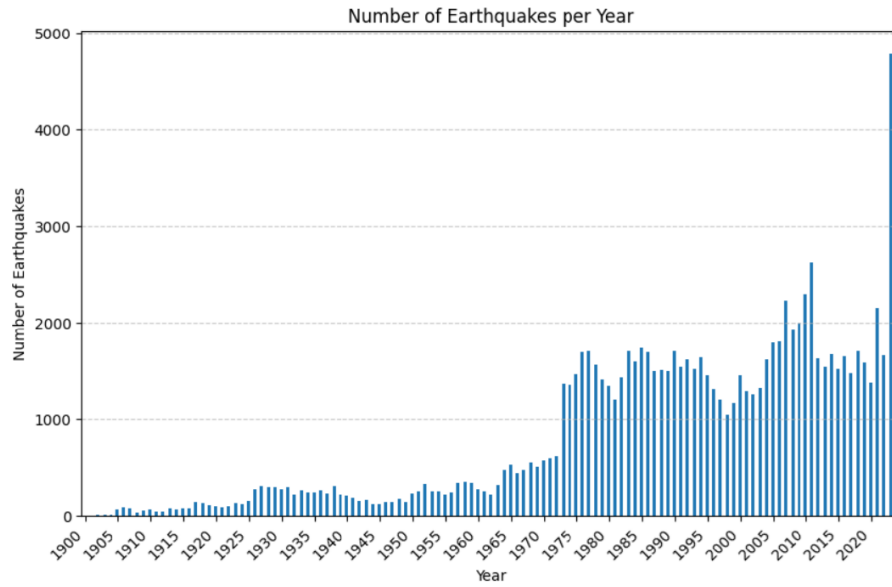


Figure 7: Number of Earthquakes

Conclusion for Number of Earthquakes: The analysis of earthquake occurrences over time reveals a complex interplay between natural seismic activity and human-induced events. While the frequency of earthquakes has shown an upward trend, this increase is largely attributed to advancements in detection technologies rather than an actual rise in seismic events. The data underscores the need for continuous monitoring and preparedness, particularly in regions prone to seismic activity.

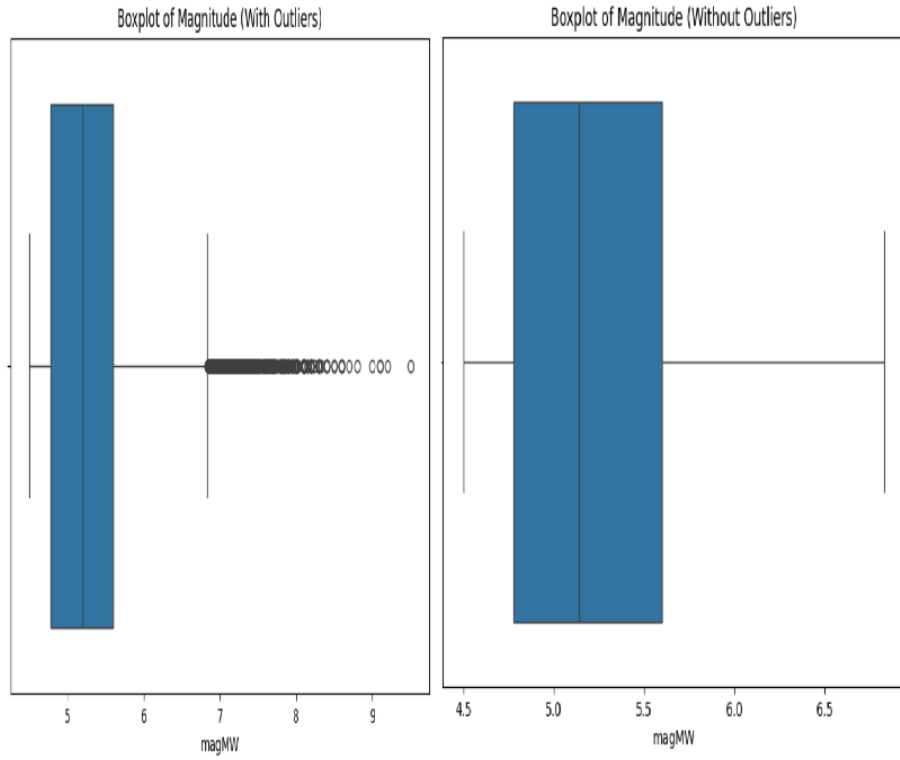


Figure 8: Boxplot of Earthquake Magnitudes

Conclusion for Boxplot: The boxplot analysis highlights the presence of outliers in earthquake magnitudes, which represent rare but significant events that can provide crucial insights into seismic behavior. These anomalies indicate important patterns or emerging trends that are vital for improving the accuracy of predictions. Eliminating them might lead to a loss of valuable data, reducing the model's ability to capture the full range of seismic activity.

3 Machine Learning Models

Models Used

- Linear Regression
- Polynomial Regression
- Random Forest Regressor
- Gradient Boosting Regressor
- XGBoost Regressor

Justification

Linear Regression: Linear Regression is selected for its simplicity and interpretability. It is less prone to overfitting due to fewer parameters and is effective for modeling linear relationships.

```
1 from sklearn.linear_model import LinearRegression
2
3 linear_model = LinearRegression()
4 linear_model.fit(X_train_scaled_quantile, y_train)
```

Polynomial Regression: Polynomial Regression extends Linear Regression to capture non-linear relationships by modeling the data as an nth-degree polynomial. This flexibility allows for controlled complexity through polynomial degree selection.

```
1 from sklearn.preprocessing import PolynomialFeatures
2 from sklearn.pipeline import make_pipeline
3
4 poly_model = make_pipeline(PolynomialFeatures(degree=2),
5                             LinearRegression())
6 poly_model.fit(X_train_scaled_quantile, y_train)
```

Random Forest Regressor: Random Forest is chosen for its robustness and ability to handle both numerical and categorical features. It builds multiple decision trees, which helps in reducing overfitting and enhancing model stability.

```
1 from sklearn.ensemble import RandomForestRegressor
2
3 model_rf = RandomForestRegressor(
4     n_estimators=625,
5     max_depth=3,
6     min_samples_split=14,
7     min_samples_leaf=8,
8     max_features='log2',
9     bootstrap=True,
10    random_state=42
11 )
```

```

12 |
13 | model_rf.fit(X_train_scaled_quantile, y_train)

```

Gradient Boosting Regressor: Gradient Boosting Regressor (GBR) is selected for its high predictive accuracy, especially on structured datasets. It builds trees sequentially, where each new tree attempts to correct errors made by previous ones. However, it can be prone to overfitting if not tuned properly.

```

1 | # Instantiate and train the model with modified hyperparameters to
  |   reduce overfitting
2 | model = GradientBoostingRegressor(
3 |     n_estimators=625,
4 |     # Increased number of trees for stability
5 |     learning_rate=0.05,
6 |     # Lowered learning rate for better generalization
7 |     max_depth=3,
8 |     # Reduced max depth to avoid overfitting
9 |     min_samples_split=14,
10 |    # Increased min samples for a split to smooth model
11 |    min_samples_leaf=8,
12 |    # Ensures leaves are not too small
13 |    subsample=0.8,
14 |    # Use 80% of data per tree to reduce overfitting
15 |    max_features='log2',
16 |    # Limit features to reduce overfitting
17 |    random_state=42,          # For reproducibility
18 | )
19 |
20 | model.fit(X_train_scaled_quantile, y_train)
21 | y_train_pred = model.predict(X_train_scaled_quantile)
22 | y_test_pred = model.predict(X_test_scaled_quantile)
23 | evaluate_model(model, X_train_scaled_quantile, y_train,
  |   X_test_scaled_quantile, y_test)

```

XGBoost Regressor: XGBoost is selected for its speed and performance, particularly in handling large datasets with missing values. Its built-in regularization techniques help reduce overfitting, making it suitable for competitive machine learning tasks. The hyperparameters are tuned to optimize model performance and prevent overfitting.

```

1 | from xgboost import XGBRegressor
2 |
3 | xgb_model = XGBRegressor(
4 |     n_estimators=653,
5 |     learning_rate=0.05,
6 |     max_depth=3,
7 |     min_child_weight=8,
8 |     gamma=0.1,
9 |     subsample=0.8,
10 |    colsample_bytree=0.8,
11 |    reg_alpha=0.15,
12 |    reg_lambda=1,
13 |    random_state=42,

```

```

14     n_jobs=-1,
15     early_stopping_rounds=40,
16     eval_metric='rmse'
17 )
18
19 xgb_model.fit(X_train_scaled_quantile, y_train, eval_set=[(
    X_test_scaled_quantile, y_test)], verbose=False)

```

3.1 Model Implementation

Training and Testing

The dataset was divided into training and testing subsets, with 80 percent of the data allocated for training and 20 percent reserved for testing. This approach was particularly important for our earthquake prediction model, as it leverages historical seismic data to forecast future earthquake magnitudes. In this context, the initial years of data were used for training the model, allowing it to learn patterns and relationships inherent in the historical data. The subsequent years were set aside for testing, enabling us to evaluate the model's predictive performance on unseen data.

This method not only ensures that the model is trained on a robust and representative dataset but also aligns with the temporal nature of earthquake occurrences. By using the most recent data for testing, we can assess the model's ability to generalize and make accurate predictions about earthquake magnitudes for the next five years. This forward-looking approach is critical in seismic research, as it allows for timely and informed decision-making in disaster preparedness and risk management. Furthermore, by validating the model on future data, we enhance its reliability and credibility, which is essential for stakeholders who rely on accurate predictions for safety and planning purposes.

Hyperparameter Tuning

Hyperparameter tuning optimizes machine learning models by finding the best configuration parameters that weren't automatically learned during training. It helps improve model performance, generalization, and prevents overfitting by systematically exploring different settings to achieve the most accurate and reliable predictive results.

Hyperparameter tuning is performed using both **RandomizedSearchCV** and **BayesSearchCV** to optimize model performance across different algorithms.

RandomizedSearchCV: RandomizedSearchCV samples a given number of candidates from a parameter space, which can be more efficient than exhaustive search methods like GridSearchCV.

```

1 from sklearn.model_selection import RandomizedSearchCV
2 from sklearn.ensemble import GradientBoostingRegressor
3

```

```

4 # Define the parameter distribution
5 param_dist = {
6     'n_estimators': [500, 600, 625, 650, 700],
7     'learning_rate': [0.01, 0.03, 0.05, 0.07, 0.1],
8     'max_depth': [2, 3, 4],
9     'min_samples_split': [10, 14, 18],
10    'min_samples_leaf': [4, 8, 12],
11    'subsample': [0.7, 0.8, 0.9],
12    'max_features': ['sqrt', 'log2', None],
13 }
14
15 # Initialize the model
16 model = GradientBoostingRegressor(random_state=42)
17
18 # RandomizedSearchCV
19 random_search = RandomizedSearchCV(
20     estimator=model,
21     param_distributions=param_dist,
22     n_iter=50,
23     cv=5,
24     scoring='r2',
25     random_state=42,
26     n_jobs=-1
27 )
28
29 # Fit the model
30 random_search.fit(X_train_scaled_quantile, y_train)
31
32 # Print the best parameters
33 print("Best parameters from RandomizedSearchCV:", random_search.
      best_params_)

```

BayesSearchCV: BayesSearchCV utilizes Bayesian optimization to search for the best hyperparameters, which can be more efficient than GridSearchCV, especially when dealing with a large search space.

```

1 from skopt import BayesSearchCV
2 from sklearn.ensemble import GradientBoostingRegressor
3
4 # Define the search space
5 search_spaces = {
6     'n_estimators': (500, 700),
7     'learning_rate': (0.01, 0.1, 'log-uniform'),
8     'max_depth': (2, 4),
9     'min_samples_split': (10, 18),
10    'min_samples_leaf': (4, 12),
11    'subsample': (0.7, 0.9),
12    'max_features': ['sqrt', 'log2', None],
13 }
14
15 # Initialize the model
16 model = GradientBoostingRegressor(random_state=42)
17
18 # Perform Bayesian Optimization
19 bayes_search = BayesSearchCV(estimator=model, search_spaces=
      search_spaces, n_iter=20, cv=3, scoring='r2', n_jobs=-1,

```

```

20     random_state=42)
21     bayes_search.fit(X_train_scaled_quantile, y_train)
22     # Print the best parameters
23     print("Best parameters from BayesSearchCV:", bayes_search.
          best_params_)

```

3.2 Manually Hypertuning

We achieved the best hyperparameters through manual tuning, using the initial results from the Randomized model as a foundation. By deeply understanding the significance of each hyperparameter, we carefully adjusted settings specifically for the Gradient Boosting model. This involved fine-tuning parameters like learning rate, number of boosting stages, maximum depth, and subsample ratio. Through this targeted approach, we were able to significantly enhance the model's performance, achieving a better balance between accuracy, bias, and variance. This hands-on method allowed us to leverage insights from the randomized baseline while optimizing the Gradient Boosting model to achieve superior predictive accuracy.

The **Randomized model** demonstrates strong performance, with relatively low error rates, indicating a good fit to both the training and test data. It captures a moderate amount of variance in the data, which reflects its ability to explain the underlying patterns during training. Although there's a slight drop in performance on unseen data, it still maintains a balance between overfitting and underfitting. The model shows minimal bias, meaning its predictions are generally close to actual values, and its variance remains within an acceptable range. This leads to a high level of accuracy within the 0.5 magnitude threshold, making it a reliable choice for this prediction task.

On the other hand, the **Bayesian model** shows slightly higher error rates, suggesting it may not fit the data as well as the Randomized model. It captures less variance, particularly in the test set, indicating limited explanatory power. The model tends to overestimate predictions slightly but remains generally consistent in its bias. However, it has low variance, which suggests it might be too rigid and could be underfitting the data. This is further reflected in its drop in accuracy on the test set, showing that it may struggle to generalize to new, unseen data compared to the Randomized model. Overall, the Randomized model outperforms the Bayesian approach in terms of flexibility and predictive accuracy.

Feature Selection and Extraction

In the context of our earthquake prediction model, we considered various techniques for feature selection and extraction, such as Recursive Feature Elimination (RFE), Principal Component Analysis (PCA), and Linear Discriminant Analysis (LDA). These methods are commonly employed to enhance model performance by identifying the most relevant features, reducing dimensionality, and improving interpretability.

However, given that our dataset consisted of only five columns, we determined that further reduction of features would be unnecessary and potentially detrimental to the model's performance. With such a limited number of features, each variable plays a significant role in contributing to the model's predictive capability. Reducing the feature set could lead to the loss of valuable information, ultimately impairing the model's ability to accurately predict earthquake magnitudes.

Thus, we opted to retain all available features in our analysis. This decision not only preserves the integrity of the data but also ensures that the model can leverage all relevant information for making accurate predictions. By maintaining the complete feature set, we aim to maximize the model's performance while ensuring that it remains interpretable and effective in its predictions.

4 Experimental Setup

4.1 Tools and Technologies

- **Programming Language:** Python 3.8
- **Core Data Science Libraries:**
 - **NumPy:** Numerical computing and array operations
 - **Pandas:** Data manipulation and analysis
 - **Scikit-learn:** Machine learning algorithms and model evaluation
 - **XGBoost:** Advanced gradient boosting framework
- **Visualization Libraries:**
 - **Matplotlib:** Comprehensive plotting
 - **Seaborn:** Statistical data visualization

4.2 Computational Environment

- **Platform:** Google Colab
- **Computational Resources:**
 - Cloud-based Jupyter notebook environment
 - GPU: Tesla T4
 - RAM: 12 GB

4.3 Evaluation Metrics

4.3.1 Quantitative Performance Metrics

1. Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_{\text{true}} - y_{\text{predicted}})^2 \quad (1)$$

MSE is the average of the squared differences between the actual values (y_{true}) and the predicted values ($y_{\text{predicted}}$).

- **Lower MSE:** Indicates that the model's predictions are close to the actual values, implying better performance.
- **Higher MSE:** Suggests larger prediction errors, indicating the model is not fitting the data well.

2. R-squared (R^2):

$$R^2 = 1 - \frac{\sum (y_{\text{true}} - y_{\text{predicted}})^2}{\sum (y_{\text{true}} - \bar{y})^2} \quad (2)$$

R^2 measures the proportion of the variance in the dependent variable that is predictable from the independent variables.

- $R^2 = 1$: Perfect model fit; the predictions match the actual values exactly.
- $R^2 = 0$: The model performs no better than simply predicting the mean of the actual values.
- $R^2 < 0$: Indicates that the model is worse than predicting the mean, which often happens with overfitting or irrelevant features.

4.3.2 Model Complexity Metrics

- **Bias:** Bias measures the systematic error in predictions, indicating whether the model consistently overestimates or underestimates the actual values. It's calculated as the average difference between predicted and actual values.
 - Bias approx 0: Indicates the model's predictions are centered around the actual values, showing good accuracy.
 - Positive Bias: The model tends to overestimate the actual values.
 - Negative Bias: The model tends to underestimate the actual values.
- **Variance:** Variance measures the variability of model predictions. It reflects how sensitive the model is to changes in the training data.
 - High Variance: Indicates overfitting, where the model is too sensitive to small fluctuations in the training data and may perform poorly on new, unseen data.
 - Low Variance: Suggests underfitting, where the model is too rigid and fails to capture important patterns in the data.
- **Accuracy (Within a Threshold):** This accuracy metric is specific to regression models where the goal is to predict values within a certain threshold of the actual value. For example, predicting earthquake magnitude within ± 0.5 of the actual value.
 - **Higher Accuracy:** Indicates that a larger proportion of predictions fall within the specified range, showing that the model is reliable for practical use.
 - **Lower Accuracy:** Shows that many predictions are outside the acceptable error range, indicating poor model performance.

4.3.3 Bias-Variance Tradeoff

- **High Bias, Low Variance:** Leads to underfitting. The model is too simple to capture the underlying data patterns (e.g., Linear Regression in our case).
- **Low Bias, High Variance:** Leads to overfitting. The model captures noise in the training data, resulting in poor performance on new data (e.g., Polynomial Regression with higher degrees).
- **Balanced Bias and Variance:** Provides the best generalization to new data (e.g., Gradient Boosting and XGBoost models).

4.4 Accuracy Threshold Rationale

4.4.1 Why 0.5 Magnitude Accuracy Matters

- **Critical Implications:**
 - Small magnitude differences significantly impact earthquake severity
 - Precise predictions crucial for:
 - * Emergency response planning
 - * Disaster management
 - * Public safety infrastructure
- **Practical Significance:**
 - 0.5 magnitude difference can represent:
 - * 10x energy release difference
 - * Substantial changes in potential damage
 - * Varied emergency response strategies

5 Results and Discussions

Model Evaluation

Training Metrics

Model	Training MSE	R ² Score	Bias	Variance	Accuracy (within 0.5 mag)
Linear Regression	0.3107	0.0928	0.0000	0.0318	62.94%
Polynomial Regression (2)	0.2901	0.1528	0.0000	0.0523	67.82%
Polynomial Regression (3)	0.2827	0.1745	0.0000	0.0598	70.03%
Polynomial Regression (4)	0.2685	0.2158	0.0000	0.0739	71.64%
Polynomial Regression (5)	0.2589	0.4383	-0.0002	0.1417	81.55%
Random Forest	0.2245	0.3443	0.0000	0.0850	80.12%
Gradient Boosting	0.1918	0.4399	0.0001	0.1400	81.48%
XGBoost	0.1924	0.4383	-0.0002	0.1417	81.55%

Table 1: Training Metrics for Various Models

Test Metrics

Model	Test MSE	R ² Score	Bias	Variance	Accuracy (within 0.5 mag)
Linear Regression	0.6598	-1.6155	-0.6289	0.0077	47.99%
Polynomial Regression (2)	0.3990	-0.5818	0.3387	0.0469	47.44%
Polynomial Regression (3)	0.3112	-0.2338	0.1171	0.0635	62.49%
Polynomial Regression (4)	0.2794	-0.1074	-0.0853	0.0212	77.50%
Polynomial Regression (5)	0.1924	0.0651	-0.0507	0.0167	79.66%
Random Forest	0.2542	-0.0079	0.0462	0.0027	70.13%
Gradient Boosting	0.2358	0.0653	-0.0604	0.0164	79.91%
XGBoost	0.2358	0.0651	-0.0507	0.0167	79.66%

Table 2: Test Metrics for Various Models

5.1 Error Analysis

Linear Regression Performance

- Linear Regression performs poorly, especially on the test set, with low accuracy within the 0.5-magnitude threshold.
- The negative R² score on the test data suggests that the model is unable to capture even basic trends.

- Predictions are, on average, worse than the mean value.
- A significantly higher MSE on the test set compared to the training set indicates that the model does not generalize well to new data.
- This implies a fundamental inability of Linear Regression to handle the complex, non-linear relationships in seismic data.
- Due to its limited predictive capability, Linear Regression is insufficient for this task.
- Moving to Polynomial Regression is a logical next step, as it introduces non-linear terms that may capture more complex patterns.

-

Polynomial Regression Performance

- Increasing polynomial degrees reduces bias but increases variance.
- Degrees 3 and 4 strike a better balance, with Degree 4 achieving the highest test threshold accuracy of 77.50%.
- MSE decreases as the degree increases, showing better fitting to the data.
- However, Degree 5 starts showing overfitting as evidenced by increased test MSE compared to Degree 4.
- Though improvements are seen, negative R^2 scores for all models on the test set indicate these models struggle to predict new data accurately, which is the greatest problem.
- There is a need for models that can maintain complexity without overfitting.
- The noticeable gap between training and test accuracy suggests slight overfitting, as Random Forest tends to fit noise when using complex data.
- Lower MSE values compared to Polynomial Regression indicate better generalization, but the slight overfitting results in a negative R^2 score.

Random Forest Performance

- Random Forest provides a noticeable improvement in predictive accuracy but struggles with overfitting.
- Exploring boosting techniques like Gradient Boosting is a promising next step, as they are designed to sequentially improve upon errors from prior models.

Gradient Boosting Performance

- Gradient Boosting achieves high accuracy within the 0.5-magnitude threshold on both training and test data, indicating good generalization to unseen data.
- The positive R^2 score on the test set suggests a reliable fit, making it the first model so far that accurately captures the data's complex structure without significant overfitting.
- This model achieves the lowest MSE among all tested models, indicating a superior fit to the data.
- The small variance between training and test metrics also indicates a well-balanced model with controlled complexity.

XGBoost Performance

- XGBoost performs similarly to Gradient Boosting, with nearly identical metrics across all evaluation measures.
- Its accuracy within the 0.5-magnitude threshold is comparable, with slightly higher training accuracy but a minor drop in test accuracy.
- This suggests that while XGBoost is effective, Gradient Boosting generalizes slightly better to unseen data.

Conclusion

- Both Gradient Boosting and XGBoost achieve high accuracy thresholds within 0.5 magnitude, making them the top choices.
- Gradient Boosting, with its marginally better test accuracy, is the optimal model for this dataset.

5.2 Graphs

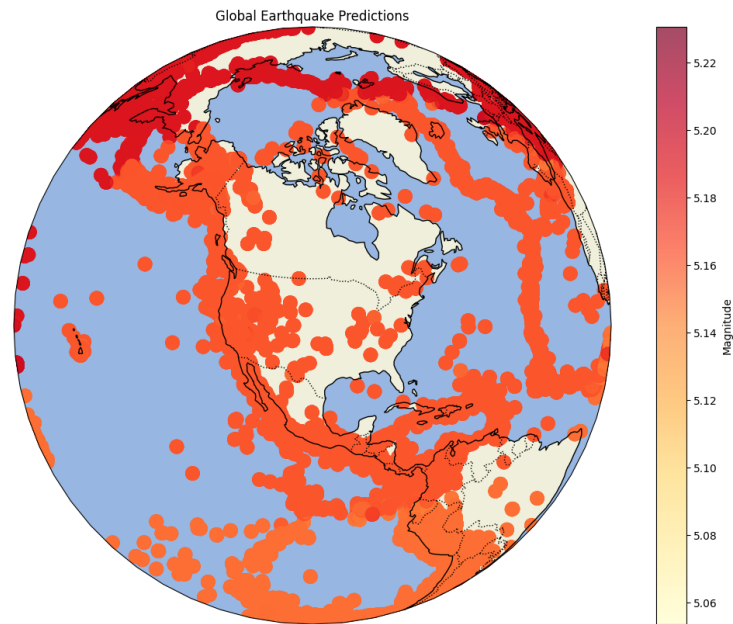


Figure 9:

The colors are mostly there for the sake of completeness. Taking a look at the actual range shows not that much variation in the model's prediction. This could be due to several reasons in the model's training. Eg. Lower magnitude earthquakes happen all the time everywhere but our dataset consists only of those that are 5 and above. Training our model on such data probably made it more prone to showing those values. Our model is as much a reflection of training data as it is of reality.

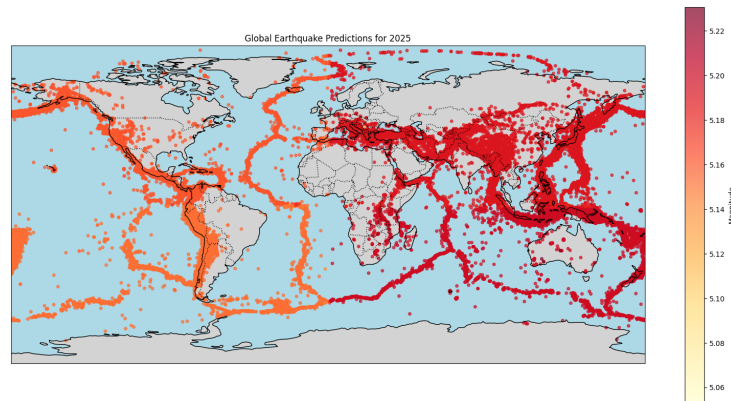


Figure 10:

Already, here we see an important quirk of our model. There is virtually no difference between the predicted earthquakes of 2025 and 2030. This means that according to our model, significant earthquake locations are more or less stable over time. In fact, to drive this point home—

Here is an image showing historical data from Wikipedia of Quake epicenters from 1963-1988. Note the stunning similarity to our graph.

Back to our model's predictions, we observe that earthquakes are particularly densely packed along a particular 'line' on the far left and right of our graph which connects countries like indonesia and japan, the entire coastline of the americas, etc.

Indeed, this is the **Pacific Ring of Fire**, a known region of tectonic activity where many of the significant earthquakes in the world have taken place for decades. The fact our model predicts this so closely is a good sign.

Our model also gets the other densely packed regions such as those right above india (due to boundary with indian plate) right

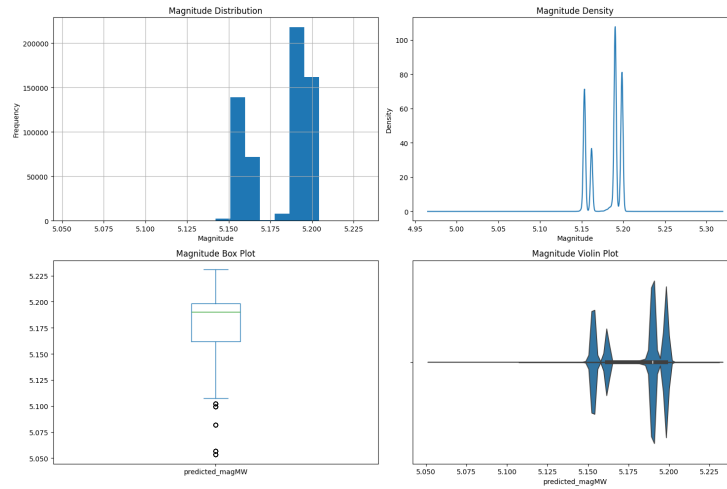


Figure 11:

In these graphs we are observing our model data itself rather than making any conclusion on what it says. Our model's magnitudes are mostly confined between 5.12-5.3 or so, with any thing below 5.12 being an outlier in our model. particular concentrations around values of 5.15, 5.19 and 5.2.

All this is due to the aforementioned effect of dataset and choice of training.

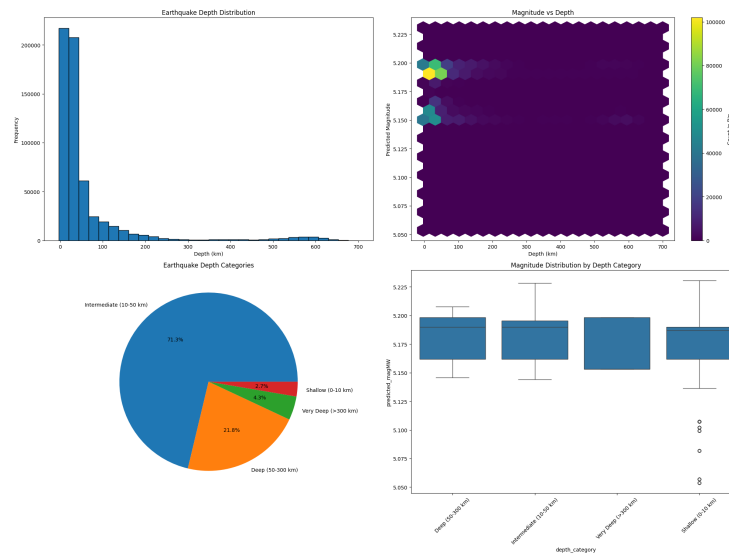


Figure 12:

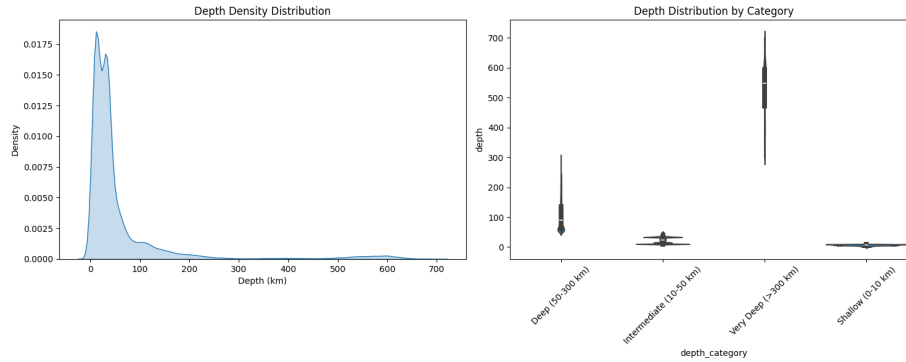


Figure 13:

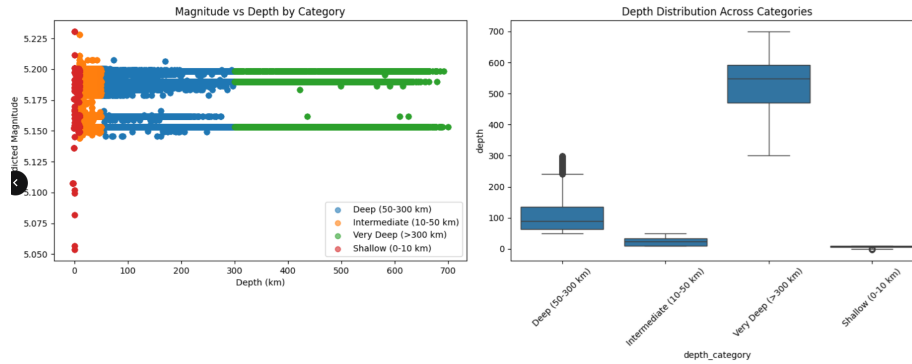


Figure 14:

Here we see the first thing that strikes us the distribution- most earthquakes happen in intermediate depths- between 10-15 km. Ones that occur deep are second most common, followed by very deep. The least common are the shallow quakes.

The hexbin plot just confirms for us that most earthquakes in our model happen at those aforementioned magnitudes, and at intermediate depth. This was just done as a way to exclude the possibility of lower magnitude peaking at different depth

We also see that acc. to model, shallow quakes are more spread out, and as we go deeper the spread in quake magnitude decreases are more spread out in terms of magnitude.

Boxplot of predicted magnitude per depth range is also present and as we can see our model predicts that magnitude does not vary with depth, which is indeed a true statement so it's good to see our model agrees with that as well.

Boxplot to see range in depth itself for depth ranges shows how widespread intermediate earthquakes are and also the few earthquakes that occur at very

deep regions. Researching a bit reveals this can be due to the aforementioned 'subduction zones' which most earthquakes at great depths are associated with, or a lot of earthquakes happening at a region whose geological activity just naturally happens at great depths.

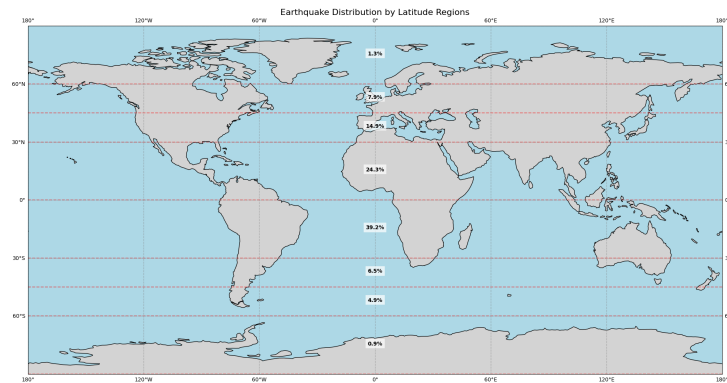


Figure 15:

Here we see are model predicts that earthquakes are more concentrated near the the equatorial regions. This is good, because it coincides with geography and science of earthquakes.

As we can see, the key is that most plate boundaries are concentrated closer to the equatorial regions at this point of earth's history. Additionally, 'subduction zones' also exist more in number closer in equatorial and tropical regions. Additionally, earth's crust is also thinner near the equator.

Our graph seems to be a well estimate in terms of earthquake placemet in the coming years at the very least, as we can see that it even accounts for subtler things like there being slightly more plate boundaries a bit to the south and the antartica right in the middle of its plate having very little earthquakes given no boundaries there.

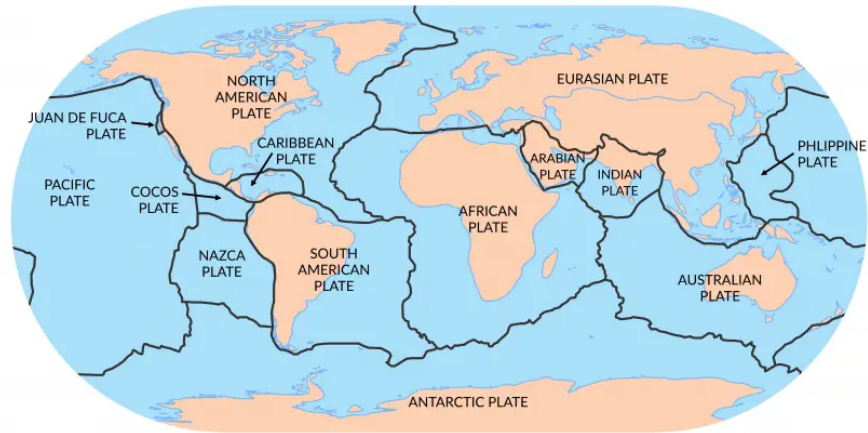


Figure 16:

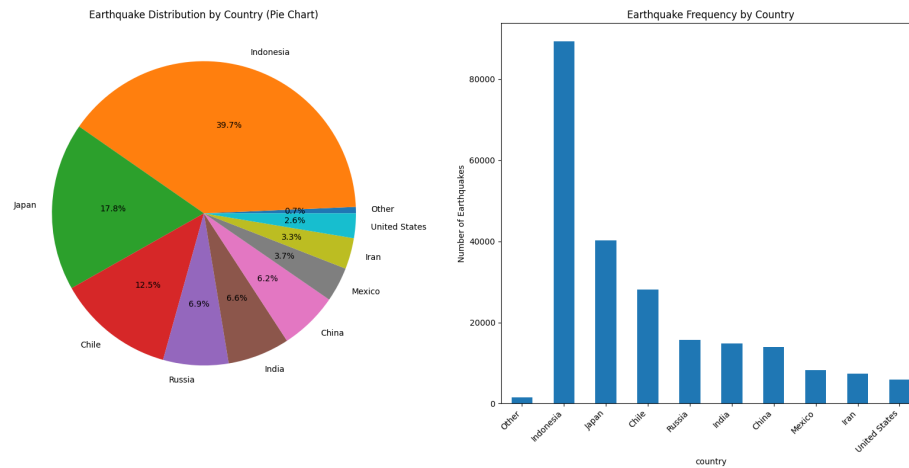


Figure 17:

A simple pie chart and bar chart shows us who will get how many quakes relative to other countries in the next 10 years.

This actually shows us an inadequacy in our model– While Indonesia being the one with most, followed by Japan is promising as it is to be expected due to their placement on the ring of fire, by all means, China should be much further ahead than it is. China is one of the countries hardest hit by quakes and there's no way it is behind India. But looking at the graph, it's clear that china is

getting a dense amount of earthquakes, but it would seem that they're lacking in number.

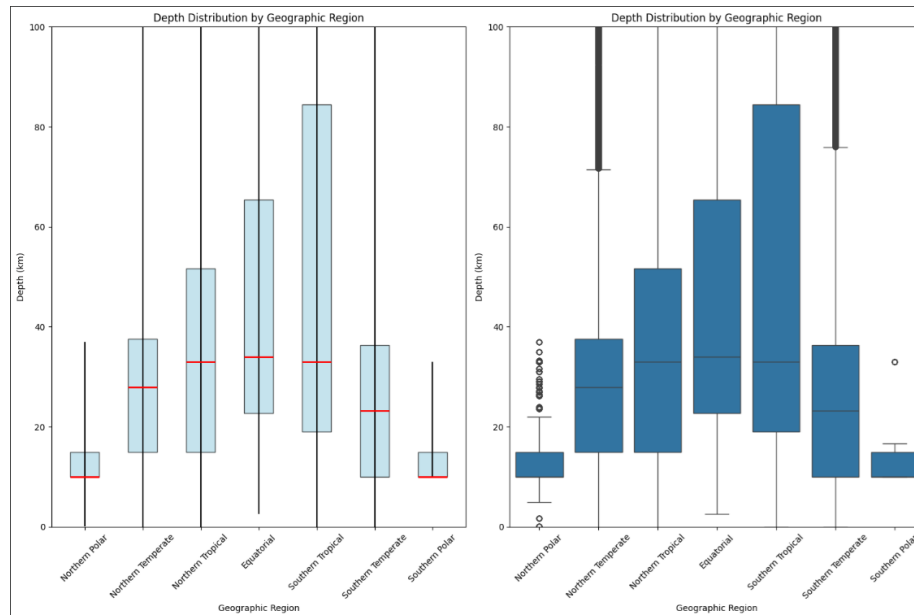


Figure 18:

Here we see boxplot distribution depth and magnitude of geographic regions. We see that polar regions have lower magnitudes and shallower depths due to lesser geological activity from plate boundaries and thinner crust at poles

6 Summary and Future Work

6.1 Summary

This project aimed to develop a predictive model for earthquake magnitudes using historical seismic data. The dataset comprised **103,973 rows and 23 columns**, including features such as **latitude, longitude, depth, and magnitude**. Various machine learning models were employed, including **Linear Regression, Polynomial Regression, Random Forest, Gradient Boosting, and XGBoost**. Hyperparameter tuning was performed using **RandomizedSearchCV and BayesSearchCV** to optimize model performance.

The analysis revealed that both **Gradient Boosting and XGBoost achieved high accuracy within a 0.5 magnitude threshold**, making them the top choices for this prediction task. **Gradient Boosting**, with its marginally better test accuracy, was identified as the optimal model for predicting earthquake magnitudes.

6.2 Findings

- **Model Performance:** Gradient Boosting and XGBoost outperformed other models in terms of accuracy and generalization to unseen data.
- **Training vs. Test Metrics:** The models demonstrated a good balance between bias and variance, indicating effective learning from the training data without significant overfitting.
- **Importance of Feature Selection:** The inclusion of relevant features such as depth and geographical coordinates was crucial for model performance.

6.3 Limitations

- **Data Size:** The dataset primarily consists of earthquake magnitudes clustered around the 5 MW range, which may limit the model's ability to generalize across a broader spectrum of magnitudes.
- **Feature Limitations:** The current model relies on only a few features (latitude, longitude, depth, year), potentially missing out on other significant predictors.
- **Potential Bias:** The dataset may have inherent biases based on the geographic regions and depth ranges included, which could affect the model's predictions.

6.4 Future Work

- **Incorporate Additional Features:** Enhance the model's performance by including geological factors (e.g., proximity to fault lines, tectonic plate

boundaries, soil composition) and environmental conditions (e.g., sea temperature anomalies, atmospheric pressure data).

- **Expand the Dataset:** Collect additional earthquake records across a wider range of magnitudes, depths, and locations to create a more balanced dataset and improve the model's robustness.
- **Explore Spatial Models:** Investigate models that account for regional variations in seismic activity, such as Geographically Weighted Regression (GWR) or spatial interpolation techniques, to capture local dependencies in seismic data.
- **Utilize Precursor Events:** Research whether precursor events or signals (e.g., foreshocks, unusual seismic patterns) can serve as additional predictors in the model, potentially improving prediction accuracy.

6.5 Timesheet

Team Member Timelines

Anish's Timeline

Task	Time Spent (hrs)
Finding dataset with appropriate features	3
Verifying the authenticity of the data	2
Data analysis and study of features	3
EDA before model training	8
Preparing documentation	4

Table 3: Anish's Timeline

Shivam's Timeline

Task	Time Spent (hrs)
Data analysis and study of features	3
Data cleaning	2
Feature reduction and impact	2
Choice of scaling	0.5
Training the model and choosing the best model	8
Hypertuning manually	4
Graph analysis	1

Table 4: Shivam's Timeline

Ruchir's Timeline

Task	Time Spent (hrs)
Data analysis and study of features	3
Test and train split of data and its impact on model	1
Hypertuning using methods like random search and Bayesian search	5
Evaluating and making graphs after model training	8
Frontend and Backend	3

Table 5: Ruchir's Timeline

7 References

Source: IRJMETS Paper - May 2023

URL: [chrome-extension://efaidnbmnnnibpcajpgclefindmkaj/https://www.irjmets.com/uploadedfiles/paper/IRJMETS Paper](chrome-extension://efaidnbmnnnibpcajpgclefindmkaj/https://www.irjmets.com/uploadedfiles/paper/IRJMETS%20Paper)

Source: Kaggle Dataset - An Analysis of Over 100 Years of Seismic Activity

URL: <https://kaggle.com/code/usamabuttar/an-analysis-of-over-100-years-of-seismic-activity>Kaggle Analysis

Source: Scikit-learn Documentation - Gradient Boosting Regression

URL: https://scikit-learn.org/1.5/auto_examples/ensemble/plot_gradient_boosting_regression.html*Scikit – learn Documentation*

Source: Kaggle Dataset - Global Earthquake Map: Interactive Insights

URL: <https://kaggle.com/code/pavankumar4757/global-earthquake-map-interactive-insights>Kaggle Insights

Source: ResearchGate - Prediction of Latitude and Longitude of Earthquakes at Global Level

URL: <https://www.researchgate.net/publication/331326965>*Prediction of Latitude and Longitude of Earthquakes at Global Level Using the Regressive Objective Regression method Research Article Corresponding author ResearchGate Article*